

---

**Security Review Report**  
**NM-0953 - Alchemix Strategies**

---



**NETHERMIND**  
**SECURITY**

(August 10, 2026)

# Contents

|          |                                                                                        |           |
|----------|----------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Executive Summary</b>                                                               | <b>2</b>  |
| <b>2</b> | <b>Audited Files</b>                                                                   | <b>3</b>  |
| <b>3</b> | <b>Summary of Issues</b>                                                               | <b>3</b>  |
| <b>4</b> | <b>System Overview</b>                                                                 | <b>4</b>  |
| 4.1      | Fluid USDC Strategy                                                                    | 4         |
| 4.2      | StakeDAO WETH Strategy                                                                 | 4         |
| <b>5</b> | <b>Risk Rating Methodology</b>                                                         | <b>5</b>  |
| <b>6</b> | <b>Issues</b>                                                                          | <b>6</b>  |
| 6.1      | [High] The main CRV yield is never claimed                                             | 6         |
| 6.2      | [Medium] Enso deallocation approves the entire RewardVault share balance to the router | 6         |
| 6.3      | [Medium] Same-transaction slippage guards are ineffective against front-running        | 7         |
| 6.4      | [Low] Fixed peg ratios drift from the pool's rising LP value                           | 7         |
| 6.5      | [Info] The allocate-path round-trip value checks are inert                             | 8         |
| <b>7</b> | <b>Documentation Evaluation</b>                                                        | <b>9</b>  |
| <b>8</b> | <b>Test Suite Evaluation</b>                                                           | <b>10</b> |
| 8.1      | Tests Output                                                                           | 10        |
| <b>9</b> | <b>About Nethermind</b>                                                                | <b>12</b> |

# 1 Executive Summary

This document presents the results of a security review conducted by [Nethermind Security](#) for Alchemix's V3 yield strategies.

Alchemix V3 is a self-repaying loan protocol in which user deposits are supplied to external yield sources and the accrued yield is used to progressively pay down the borrower's debt. Idle capital held by the protocol's underlying Morpho V2 vault (the MYT) is routed to third-party yield venues through the `AlchemistAllocator`, which allocates and deallocates funds across a set of strategy adapters. Each adapter inherits from the `MYTStrategy` base contract and exposes a uniform `allocate`, `deallocate`, and `valuation` interface, isolating the protocol from the specifics of any individual integration while enforcing per-strategy caps, slippage bounds, and a kill switch that lets the protocol bypass a compromised venue without reverting the allocator.

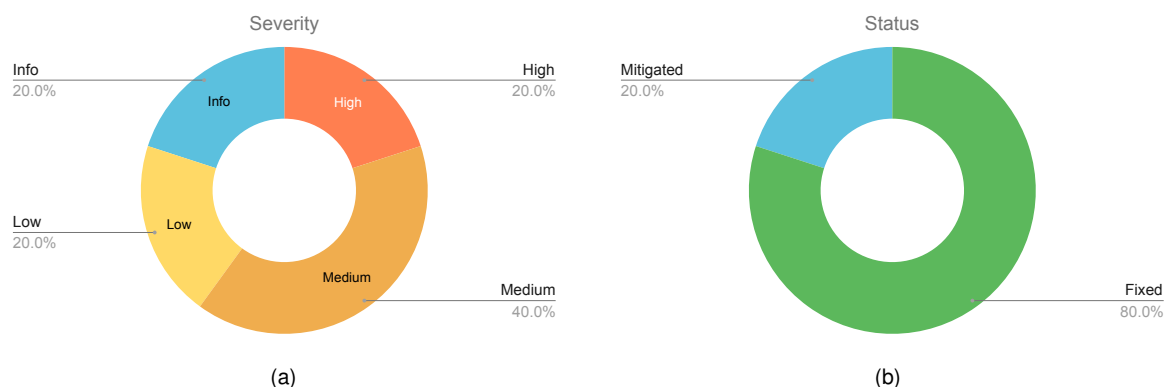
This audit focuses on the following strategies:

- **Fluid USDC strategy:** an `ERC4626Strategy` adapter that deposits the vault's USDC into Fluid's ERC-4626 USDC vault and redeems shares on withdrawal, valuing its position through the vault's `preview` functions.
- **StakeDAO WETH strategy:** a `MYTStrategy` adapter that deploys the vault's WETH into a Stake DAO `RewardVault` backed by a Curve `StableSwap` LP token, supporting both a direct Curve path and an Enso-routed swap path in each direction.

The audit comprises 295 lines of Solidity code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report five points of attention, where one is classified as High, two are classified as Medium, one is classified as Low, and one is classified as Informational severity. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 4 presents the System Overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.



**Fig. 1: Distribution of issues: Critical (0), High (1), Medium (2), Low (1), Undetermined (0), Informational (1), Best Practices (0). Distribution of status: Fixed (4), Acknowledged (0), Mitigated (1), Unresolved (0)**

## Summary of the Audit

|                                 |                                                                                                          |
|---------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>Audit Type</b>               | Security Review                                                                                          |
| <b>Initial Report</b>           | July 27, 2026                                                                                            |
| <b>Final Report</b>             | August 10, 2026                                                                                          |
| <b>Initial Commit</b>           | <b>StakeDAO Strategy:</b> <a href="#">da06df5</a><br><b>Fluid USDC Strategy:</b> <a href="#">c3ae9d1</a> |
| <b>Final Commit</b>             | <b>StakeDAO Strategy:</b> <a href="#">37f458e</a><br><b>Fluid USDC Strategy:</b> <a href="#">c3ae9d1</a> |
| <b>Documentation Assessment</b> | High                                                                                                     |
| <b>Test Suite Assessment</b>    | High                                                                                                     |

## 2 Audited Files

|   | Contract                                                | LoC        | Comments  | Ratio       | Blank     | Total      |
|---|---------------------------------------------------------|------------|-----------|-------------|-----------|------------|
| 1 | <a href="#">strategies/ERC4626Strategy.sol</a>          | 60         | 5         | 8.3%        | 15        | 80         |
| 2 | <a href="#">src/strategies/StakeDAOWETHStrategy.sol</a> | 235        | 10        | 4.3%        | 59        | 304        |
|   | <b>Total</b>                                            | <b>295</b> | <b>15</b> | <b>5.1%</b> | <b>74</b> | <b>384</b> |

## 3 Summary of Issues

|   | Finding                                                                                       | Severity | Update    |
|---|-----------------------------------------------------------------------------------------------|----------|-----------|
| 1 | <a href="#">The main CRV yield is never claimed</a>                                           | High     | Fixed     |
| 2 | <a href="#">Enso deallocation approves the entire RewardVault share balance to the router</a> | Medium   | Fixed     |
| 3 | <a href="#">Same-transaction slippage guards are ineffective against front-running</a>        | Medium   | Fixed     |
| 4 | <a href="#">Fixed peg ratios drift from the pool's rising LP value</a>                        | Low      | Mitigated |
| 5 | <a href="#">The allocate-path round-trip value checks are inert</a>                           | Info     | Fixed     |

## 4 System Overview

The Alchemix MYT protocol is a yield aggregation system built on the Morpho V2 Vault infrastructure. The protocol enables users to deposit assets into a vault that allocates funds across multiple yield-generating strategies, maximizing returns while managing risk through a cap and risk classification system. The protocol is composed of the following core components:

- **MYT Vault:** The central ERC-4626 compliant vault that serves as the liquidity hub. It manages user deposits, tracks total assets, and orchestrates the movement of funds into various adapters.
- **MYT Strategies:** Modular, protocol-specific adapters that connect the MYT Vault to external DeFi protocols. Each strategy encapsulates the logic for depositing, withdrawing, and claiming rewards from its specific target.
- **Alchemist Allocator:** A permissioned allocation manager that has the allocator role in the vault. It validates all allocation requests against strict risk controls and cap limits before interacting with the vault.
- **Alchemist Curator:** A governance and configuration manager responsible for onboarding strategies and adjusting allocation caps.
- **Risk Classifier:** The `AlchemistStrategyClassifier` contract categorizes strategies into specific risk levels (Low, Medium, High). It enforces global and local allocation caps based on these risk profiles.

This audit is scoped to the strategy adapter layer, and in particular to the strategies described in the following sub-sections.

### 4.1 Fluid USDC Strategy

The Fluid USDC strategy is a deployment of the `ERC4626Strategy` adapter that connects the MYT to Fluid's ERC-4626 USDC vault. At construction it records the MYT's underlying asset and the target vault address, and requires that the target vault's asset matches the MYT's asset.

The strategy is exercised through the base contract's two vault-facing entry points:

- `allocate`: called by the MYT to deploy assets. For the `direct` action it forwards to the ERC-4626 allocation hook, which approves the target vault and deposits the supplied amount, minting Fluid vault shares to the strategy.
- `deallocate`: called by the MYT to recover assets. For the `direct` action it forwards to the ERC-4626 deallocation hook, which withdraws any shortfall between the requested amount and the strategy's idle balance from the Fluid vault, then approves the vault to pull the requested amount back.

Valuation is reported through `_totalValue`, which prices the strategy's Fluid share balance with the vault's `previewRedeem` and adds any idle assets held directly by the strategy.

### 4.2 StakeDAO WETH Strategy

The StakeDAO WETH strategy is a `MYTStrategy` adapter that deploys the MYT's WETH into a Stake DAO `RewardVault` and exits back to WETH. The `RewardVault` is an ERC-4626 vault whose underlying asset is a Curve `StableSwap` LP token, so a WETH position is held two layers deep: WETH is first supplied to a Curve pool to mint LP, and the LP is then deposited into the `RewardVault` for shares. At construction the adapter records the MYT's WETH asset, the `RewardVault`, the Curve pool, and an Enso router address, and requires that the `RewardVault`'s asset equals the configured Curve pool. It also stores several risk parameters:

```

1  uint256 public withdrawBufferBps;
2  uint256 public minWethPerCurveLp;
3  uint256 public minCurveLpPerWeth;
```

Here `withdrawBufferBps` is bounded below `MAX_WITHDRAW_BUFFER_BPS` (650), `minWethPerCurveLp` is a WETH-per-LP price floor, and `minCurveLpPerWeth` is an LP-per-WETH allocation floor.

The strategy supports two execution paths for each direction, selected by the action type passed to the base `allocate` and `deallocate` entry points:

- `allocate`, `direct` action: supplies WETH to the Curve pool via `add_liquidity`, then deposits the minted LP into the `RewardVault`. The minimum LP out is the larger of the slippage-adjusted quote from `calc_token_amount` and the static `minCurveLpPerWeth` floor.
- `allocate`, `swap` action: routes WETH directly into `RewardVault` shares through a single Enso route, bounded by a minimum-shares floor derived from the Curve quote and the `RewardVault` preview, with an additional LP floor check on the shares received.
- `deallocate`, `direct` action: covers any shortfall beyond idle WETH by redeeming `RewardVault` shares back to LP and calling `remove_liquidity_one_coin` on the Curve pool to obtain WETH. The LP spent is bounded by `minWethPerCurveLp` through a maximum-LP check, and the withdraw buffer is applied when estimating the LP required.
- `deallocate`, `swap` action: routes `RewardVault` shares back to WETH through a single Enso route, with the LP effectively spent bounded by the maximum-LP check.

Valuation is reported through `_totalValue`, which converts the strategy's `RewardVault` shares to LP via `convertToAssets` and prices that LP at the Curve pool's virtual price, adding any idle WETH:

$$lpValue = convertToAssets(shares) \times \frac{get\_virtual\_price()}{10^{18}} \quad (1)$$

## 5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

**Likelihood** measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

**Impact** is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

|        |              | Severity Risk       |              |              |
|--------|--------------|---------------------|--------------|--------------|
| Impact | High         | Medium              | High         | Critical     |
|        | Medium       | Low                 | Medium       | High         |
|        | Low          | Info/Best Practices | Low          | Medium       |
|        | Undetermined | Undetermined        | Undetermined | Undetermined |
|        |              | Low                 | Medium       | High         |
|        |              | Likelihood          |              |              |

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

## 6 Issues

### 6.1 [High] The main CRV yield is never claimed

**File(s):** [StakeDAOETHStrategy.sol](#)

**Description:** Per [Stake DAO docs](#), a RewardVault has two separate reward channels: extra incentive tokens distributed by the vault itself (claimed via `rewardVault.claim`), and the principal reward (CRV), which is accounted in a separate Accountant contract and claimed via `Accountant.claim`.

`_claimRewards` only uses the `RewardVault` path:

```

1  function _claimRewards(address token, bytes memory quote, uint256 minAmountOut) internal override returns (uint256
   → rewardsClaimed) {
2      if (rewardVault.earned(address(this), token) == 0) return 0; // CRV is not a vault reward token -> returns 0 ->
   → early exit
3      ...
4      rewardVault.claim(rewardTokens, address(this)); // only covers the vault's extra tokens
5      ...
6  }
```

The strategy has no code path to reach the Accountant. For CRV, `rewardVault.earned(this, CRV)` returns 0, so `_claimRewards` exits early and claims nothing. CRV therefore sits unharvested to the strategy in the gauge/Accountant indefinitely. Note also the vault's CHECKPOINT reward policy: CRV must be harvested before it can be claimed.

**Recommendation(s):** Consider adding an Accountant integration to claim the main rewards using the `Accountant.claim` function.

**Status:** Fixed

**Update from the client:** Agreed, commit : [2aed4030e5064dc1be61dafaf1d7ca604783f95c](#)

### 6.2 [Medium] Enso deallocation approves the entire RewardVault share balance to the router

**File(s):** [StakeDAOETHStrategy.sol](#)

**Description:** The Enso deallocation path approves the router for the strategy's full share balance, though only enough to cover shortfall is needed:

```

1  function _deallocate(uint256 amount, bytes memory ensoCalldata) internal override returns (uint256) {
2      // ...
3      uint256 shares = rewardVault.balanceOf(address(this));
4      require(shares > 0, "No RewardVault shares");
5      // @audit ensoRouter is approved to spend the full balance
6      TokenUtils.safeApprove(address(rewardVault), ensoRouter, shares);
7      _ensoRoute(address(weth), shortfall, ensoCalldata);
8      TokenUtils.safeApprove(address(rewardVault), ensoRouter, 0);
9      // ...
10 }
```

An incorrect or compromised route can therefore pull far more shares than the withdrawal requires. A post-check (`lpSpent <= _maxLpForWeth(wethReceived)`) does bound value conservation at the floor price, and the allowance is reset to 0 in the same call, so this is not a direct drain. But the full approval still bounds only value, not scope: it lets a single deallocation spend the entire position instead of just the slice needed for shortfall.

**Recommendation(s):** Consider approving only the shares the route should need.

**Status:** Fixed

**Update from the client:** Agreed, commit : [da60b1bebed08499104efa2236dede68066cbc9a](#)

## 6.3 [Medium] Same-transaction slippage guards are ineffective against front-running

**File(s):** StakeDAOWETHStrategy.sol

**Description:** Before depositing WETH into the Curve pool, `_allocate` asks the pool what it should expect to receive (`calc_token_amount`) and then refuses anything worse than `slippageBPS` below that quote. However, this quote is read from the pool at execution time, inside the same transaction as the deposit, so it reflects whatever price the pool is at after any front-run, not a fair, independent reference price.

```

1  function _allocate(uint256 amount) internal override returns (uint256) {
2      _ensureIdleBalance(address(weth), amount);
3      uint256 sharesBefore = rewardVault.balanceOf(address(this));
4
5      uint256[] memory amounts = new uint256[](2);
6      amounts[uint256(uint128(WETH_COIN_INDEX))] = amount;
7      // @audit spot quote, read after any front-run
8      uint256 expectedLp = curvePool.calc_token_amount(amounts, true);
9      uint256 minLpOut = _minAmountAfterSlippage(expectedLp);
10     uint256 floorLpOut = _minLpForWeth(amount);
11     if (floorLpOut > minLpOut) minLpOut = floorLpOut;
12
13     TokenUtils.safeApprove(address(weth), address(curvePool), amount);
14     // @audit executes at the same manipulated price
15     uint256 lpMinted = curvePool.add_liquidity(amounts, minLpOut, address(this));
16     ...
17 }

```

`calc_token_amount` uses the same calculation as `add_liquidity`, so in one transaction the quote and the actual mint agree and `minLpOut` is always satisfied. front-running and imbalancing the pool lowers `expectedLp`, `minLpOut` drops with it, and the bad deal still passes, the `slippageBPS` check cannot fire.

Note that this applies to both `_allocate` functions and `_deallocate(uint256)`.

**Recommendation(s):** Consider relying on a reference a swap cannot move, to calculate the minimum out. For example, passing a caller-computed value through the action data, quoted off-chain. Alternatively, `get_virtual_price()` can be used with an extra tolerance window.

**Status:** Fixed

**Update from the client:** Since passing a caller-computed value for the direct path would break the existing api (of already deployed strategies), the alternative recommendation is being used i.e to use `get_virtual_price()`. The existing offchain paths additionally use a caller supplied `minOut` value in the offchain request/calldata. Commit : [f2ae76fc7827672ebc6ca0bbfcc5bb8038f19284](https://github.com/NethermindSecurity/alchemy-strategies/commit/f2ae76fc7827672ebc6ca0bbfcc5bb8038f19284)

## 6.4 [Low] Fixed peg ratios drift from the pool's rising LP value

**File(s):** StakeDAOWETHStrategy.sol

**Description:** The strategy's real protection against a bad LP/WETH rate is a pair of fixed ratios held in storage: `minCurveLpPerWeth` and `minWethPerCurveLp`, applied through `_minLpForWeth` (deposit floor) and `_maxLpForWeth` (withdrawal cap):

```

1  function _minLpForWeth(uint256 wethAmount) internal view returns (uint256) {
2      return Math.mulDiv(wethAmount, minCurveLpPerWeth, PRICE_SCALE, Math.Rounding.Ceil);
3  }
4
5  function _maxLpForWeth(uint256 wethAmount) internal view returns (uint256) {
6      return Math.mulDiv(wethAmount, PRICE_SCALE, minWethPerCurveLp, Math.Rounding.Ceil) + LP_ROUNDING_TOLERANCE;
7  }

```

These ratios are constants, but the true worth of one LP token is expected to keep rising (the pool accrues swap fees and the rate-scaled coin appreciates). So the fixed ratios and the real rate drift apart over time, with opposite consequences on each side:

- Deposit (freeze): as LP appreciates, the LP received per WETH deposited falls toward the fixed minimum in `_minLpForWeth`. Once the real amount drops below it, `floorLpOut` can no longer be met and every deposit reverts, fixable only by the admin lowering `minCurveLpPerWeth`;
- Withdrawal (weakening protection): the LP actually needed per WETH withdrawn falls away from the fixed cap in `_maxLpForWeth`, so the cap becomes increasingly generous. The gap between what should be paid and what the cap allows widens;

**Recommendation(s):** Consider avoiding to hard-code the ratio, by deriving both bounds from a reference that tracks the pool's current value automatically.

**Status:** Mitigated

**Update from the client:** Also fixed in previous commit : [f2ae76fc7827672ebc6ca0bbfcc5bb8038f19284](#). Since execution bounds will be derived from `get_virtual_price()`, they track LP appreciation automatically. The existing absolute ratios are retained only as loose emergency circuit breakers.

## 6.5 [Info] The allocate-path round-trip value checks are inert

**File(s):** `StakeDAOWETHStrategy.sol`

**Description:** Both allocate paths end with a check that re-values the shares just received back into WETH via `_sharesToWeth` and compares it to a slippage-adjusted minimum. Neither check can meaningfully fire.

**Direct path (`_allocate(uint256)`):**

```
1 uint256 sharesReceived = rewardVault.balanceOf(address(this)) - sharesBefore;
2 uint256 wethValueReceived = _sharesToWeth(sharesReceived);
3 uint256 minWethValue = _minWethAfterSlippage(amount);
4 if (wethValueReceived < minWethValue) revert InvalidAmount(minWethValue, wethValueReceived);
```

`_sharesToWeth` runs `calc_withdraw_one_coin` on the LP just minted, i.e. it measures depositing WETH and immediately withdrawing the same coin at the same pool state. That round trip nets to the input minus roughly two dynamic fees regardless of pool imbalance, and the pool's dynamic fee is bounded, so `wethValueReceived` cannot drop below  $\text{amount} \times (10_{000} - \text{slippageBPS}) / 10_{000}$  under any realistic state. It never reverts, and because both legs read the same state it also catches no manipulation.

**Swap path (`_allocate(uint256, bytes)`):**

```
1 uint256 wethValueReceived = _sharesToWeth(sharesReceived);
2 uint256 minWethValue = _sharesToWeth(minSharesOut);
3 if (wethValueReceived < minWethValue) revert InvalidAmount(minWethValue, wethValueReceived);
```

Here it is strictly redundant: `_ensoRoute` already enforces `sharesReceived >= minSharesOut`, and `_sharesToWeth` is monotonic in shares, so `wethValueReceived >= minWethValue` always holds. The check can never revert.

**Recommendation(s):** Consider removing both unnecessary round-trip checks.

**Status:** Fixed

**Update from the client:** Agreed, commit: [d2c7c44f9c434307b6d2452afa747dbcf1d2ef9c](#)

## 7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

### Remarks about Alchemix Strategies documentation

The documentation for this audit consisted primarily of inline NatSpec comments within the in-scope strategy contracts. Additionally, the Alchemix team addressed all questions and concerns raised by the Nethermind Security team, providing valuable insights and a comprehensive understanding of the project's technical aspects.

## 8 Test Suite Evaluation

### 8.1 Tests Output

```
> forge test --match-contract "StakeDAOWETHStrategy|FluidARBUSDCStrategy" -vv

Ran 6 tests for src/test/DeployStakeDAOWETHStrategyScript.t.sol:DeployStakeDAOWETHStrategyScriptTest
[PASS] test_defaultParams_usesMainnetStakeDAODefaults() (gas: 43033)
[PASS] test_deployStakeDAOWETHStrategy_blocksDirectForceDeallocateByDefault() (gas: 4106607)
[PASS] test_deployStakeDAOWETHStrategy_blocksForceDeallocateSwap() (gas: 4105008)
[PASS] test_deployStakeDAOWETHStrategy_setsCoreAddressesAndDefaults() (gas: 4144465)
[PASS] test_run_fork_allocatorCanAllocateAndDeallocateStakeDAOWETHDirect() (gas: 12434952)
[PASS] test_run_fork_deploysStakeDAOWETHWithMainnetDefaults() (gas: 9430004)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 93.56ms (175.45ms CPU time)

Ran 10 tests for src/test/strategies/StakeDAOWETHStrategy.t.sol:StakeDAOWETHStrategyEnsoTest
[PASS] test_allocate_reverts_when_ens0_under_delivers_shares() (gas: 4677469)
[PASS] test_allocate_succeeds_at_exact_ens0_slippage_boundary() (gas: 4691738)
[PASS] test_allocate_with_ens0_mints_reward_vault_shares() (gas: 363300)
[PASS] test_allocate_with_ens0_reverts_below_lp_per_weth_floor() (gas: 346541)
[PASS] test_allocator_allocateWithSwap() (gas: 283275)
[PASS] test_allocator_deallocateWithSwap() (gas: 375944)
[PASS] test_deallocate_with_ens0_returns_weth() (gas: 469628)
[PASS] test_deallocate_with_ens0_reverts_below_curve_lp_floor() (gas: 446583)
[PASS] test_force_deallocate_swap_reverts() (gas: 22234)
[PASS] test_rescue_tokens_reverts_for_stakedao_position_tokens() (gas: 37312)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 2.58s (585.99ms CPU time)

Ran 26 tests for src/test/strategies/FluidARBUSDCStrategy.t.sol:FluidARBUSDCStrategyTest
[PASS] testConfig() (gas: 11969)
[PASS] test_allocator_allocate_direct(uint256) (runs: 256, : 585540, ~: 585843)
[PASS] test_allocator_deallocate_direct(uint256,uint256) (runs: 256, : 697316, ~: 698011)
[PASS] test_end_to_end_multiple_allocations_with_time_warp() (gas: 1267521)
[PASS] test_fluid_arbusdc_full_lifecycle_with_time() (gas: 599494)
[PASS] test_fluid_arbusdc_yield_accumulation() (gas: 1023280)
[PASS] test_fuzz_allocation_increases_real_assets(uint256) (runs: 256, : 591213, ~: 591516)
[PASS] test_fuzz_allocation_respects_caps(uint256) (runs: 256, : 516695, ~: 580924)
[PASS] test_fuzz_cannot_allocate_more_than_available(uint256) (runs: 256, : 448157, ~: 387799)
[PASS] test_fuzz_deallocation_decreases_real_assets(uint256,uint256) (runs: 256, : 701060, ~: 701311)
[PASS] test_fuzz_fluid_arbusdc_operations(uint256[],uint256[]) (runs: 256, : 641891, ~: 596639)
[PASS] test_fuzz_full_lifecycle_with_time_accumulation(uint256,uint256,uint256) (runs: 256, : 1390184, ~: 1460796)
[PASS] test_fuzz_multiple_allocations_deallocations(uint256[],uint8[]) (runs: 256, : 1586519, ~: 1598278)
[PASS] test_fuzz_real_assets_non_negative(uint256[],uint8[]) (runs: 256, : 1638466, ~: 1658781)
[PASS] test_fuzz_repeated_operations_stability(uint256,uint8) (runs: 256, : 4104764, ~: 3122205)
[PASS] test_fuzz_time_warp_stability(uint256,uint256,uint8) (runs: 256, : 636133, ~: 630181)
[PASS] test_fuzz_zero_operations_no_effect(uint256) (runs: 256, : 666079, ~: 666382)
[PASS] test_strategy_accumulation_over_time() (gas: 1420446)
[PASS] test_strategy_allocate_reverts_due_to_paused_allocation() (gas: 284739)
[PASS] test_strategy_allocate_reverts_due_to_zero_amount() (gas: 259108)
[PASS] test_strategy_deallocate(uint256) (runs: 256, : 595688, ~: 596329)
[PASS] test_strategy_deallocate_reverts_due_to_slippage(uint256,uint256) (runs: 256, : 408732, ~: 408800)
[PASS] test_strategy_deallocate_reverts_due_to_zero_amount() (gas: 392760)
[PASS] test_strategy_forceDeallocate_swap_reverts() (gas: 23132)
[PASS] test_strategy_forceDeallocate_unwrapAndSwap_reverts() (gas: 22913)
[PASS] test_strategy_withdrawToVault(uint256) (runs: 256, : 292074, ~: 292119)
Suite result: ok. 26 passed; 0 failed; 0 skipped; finished in 2.80s (10.97s CPU time)

Ran 45 tests for src/test/strategies/StakeDAOWETHStrategy.t.sol:StakeDAOWETHStrategyDirectTest
[PASS] testConfig() (gas: 12299)
[PASS] testFuzz_accounting_uses_reward_vault_assets_after_lp_accrual(uint256,uint256) (runs: 256, : 1529686, ~: 1529870)
[PASS] testFuzz_deallocate_uses_idle_weth_before_reward_vault_shares(uint256,uint256,uint256) (runs: 256, : 1826538, ~: 1831051)
[PASS] testFuzz_deallocate_uses_idle_weth_then_reward_vault_shares_for_shortfall(uint256,uint256,uint256) (runs: 256, : 2183461, ~: 2186592)
[PASS] testFuzz_forceDeallocate_direct_succeeds(uint256,uint256) (runs: 256, : 2136255, ~: 2142537)
[FAIL: Allocation should not exceed requested amount: 11001678067549123740 > 11000000000000000000; counterexample:
↳ args=[11000000000000000000 [1.1e19]] test_allocator_allocate_direct(uint256) (runs: 0, : 0, ~: 0)
[PASS] test_allocator_deallocate_direct(uint256,uint256) (runs: 256, : 2275288, ~: 2282896)
[PASS] test_claimRewards_can_sell_half_cvx_and_leave_remainder_in_strategy() (gas: 2388802)
[PASS] test_claimRewards_uses_real_reward_vault_and_swaps_cvx_to_weth() (gas: 2376521)
[PASS] test_direct_allocate_reverts_when_curve_manipulation_pushes_lp_output_below_floor() (gas: 781779)
[PASS] test_direct_deallocate_reverts_when_curve_lp_price_is_below_floor() (gas: 1505886)
```

```
[PASS] test_direct_deallocate_reverts_when_curve_manipulation_pushes_weth_output_below_floor() (gas: 1840925)
[PASS] test_end_to_end_multiple_allocations_with_time_warp() (gas: 14316861)
[PASS] test_force_deallocate_defaults_disabled_and_owner_can_enable() (gas: 42504)
[PASS] test_fuzz_allocation_increases_real_assets(uint256) (runs: 256, : 1797734, ~: 1801743)
[PASS] test_fuzz_allocation_respects_caps(uint256) (runs: 256, : 975709, ~: 507515)
[PASS] test_fuzz_cannot_allocate_more_than_available(uint256) (runs: 256, : 689558, ~: 480496)
[PASS] test_fuzz_deallocation_decreases_real_assets(uint256,uint256) (runs: 256, : 2278339, ~: 2285269)
[PASS] test_fuzz_full_lifecycle_with_time_accumulation(uint256,uint256,uint256) (runs: 256, : 16380293, ~: 16429484)
[PASS] test_fuzz_multiple_allocations_deallocations(uint256[],uint8[]) (runs: 256, : 13226838, ~: 13064436)
[PASS] test_fuzz_real_assets_non_negative(uint256[],uint8[]) (runs: 256, : 35793193, ~: 25777762)
[PASS] test_fuzz_repeated_operations_stability(uint256,uint8) (runs: 256, : 11996642, ~: 9188642)
[PASS] test_fuzz_time_warp_stability(uint256,uint256,uint8) (runs: 256, : 1887254, ~: 1868017)
[PASS] test_fuzz_zero_operations_no_effect(uint256) (runs: 256, : 1882514, ~: 1887803)
[PASS] test_owner_can_rescue_unsold_cvx_after_partial_reward_swap() (gas: 2404386)
[PASS] test_owner_can_set_curve_lp_floor() (gas: 23314)
[PASS] test_owner_can_set_lp_per_weth_allocation_floor() (gas: 23512)
[PASS] test_owner_can_set_withdraw_buffer_independently_from_slippage() (gas: 56907)
[PASS] test_preview_adjusted_withdraw_caps_position_at_curve_lp_floor() (gas: 1486831)
[PASS] test_set_curve_lp_floor_reverts_for_non_owner_or_zero() (gas: 21309)
[PASS] test_set_lp_per_weth_allocation_floor_reverts_for_non_owner_or_zero() (gas: 21432)
[PASS] test_set_withdraw_buffer_reverts_above_cap() (gas: 17921)
[PASS] test_strategy_accumulation_over_time() (gas: 18076253)
[PASS] test_strategy_allocate_reverts_due_to_paused_allocation() (gas: 275886)
[PASS] test_strategy_allocate_reverts_due_to_zero_amount() (gas: 250264)
[PASS] test_strategy_deallocate(uint256) (runs: 256, : 2140389, ~: 2148238)
[PASS] test_strategy_deallocate_reverts_due_to_zero_amount() (gas: 1496909)
[PASS] test_strategy_forceDeallocate_swap_reverts() (gas: 23754)
[PASS] test_strategy_forceDeallocate_unwrapAndSwap_reverts() (gas: 23535)
[PASS] test_strategy_withdrawToVault(uint256) (runs: 256, : 277242, ~: 277332)
[PASS] test_total_value_resists_direct_token_donations_to_curve_pool() (gas: 1902264)
[PASS] test_total_value_resists_imbalanced_liquidity_addition_and_removal() (gas: 1874321)
[PASS] test_total_value_resists_single_block_curve_exit_price_manipulation() (gas: 1890561)
[PASS] test_total_value_tracks_eth_plus_rate_provider_changes() (gas: 1524057)
[PASS] test_total_value_uses_reward_vault_lp_assets_and_virtual_price() (gas: 1479958)
Suite result: FAILED. 44 passed; 1 failed; 0 skipped; finished in 362.10s (1522.81s CPU time)
```

```
Ran 4 test suites in 362.12s (367.58s CPU time): 86 tests passed, 1 failed, 0 skipped (87 total tests)
```

## 9 About Nethermind

**Nethermind** is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

**Blockchain Security:** At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

**Blockchain Core Development:** Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

**DevOps and Infrastructure Management:** Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

**Cryptography Research:** At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

**Smart Contract Development & DeFi Research:** Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

**Our suite of L2 tooling:** Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

### General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

### Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.